

Client Onboarding Guide

Add a new client to the CAPI gateway in 7 steps. No code — just one database row and GoHighLevel setup.

Endpoint `https://volt-capi-getaway-six.vercel.app/api/capi/{CLIENT_ID}`

How it works

1 · Landing page

Meta Pixel sets the `_fbp` cookie; capture script fills hidden fields.

2 · Form submitted

GHL saves name, phone, zip + the hidden fields to the contact.

3 · Workflow webhook

Fires to your gateway with all the data.

4 · Gateway → Meta

Hashes the data and sends the Lead server-side.

The Lead fires server-side on form submission. There is no browser Lead event — never add `fbq('track', 'Lead')`, it double-counts.

1 Get Meta credentials

In the client's Events Manager → their dataset

- ☐ Copy the **Dataset (Pixel) ID**.
- ☐ Settings → **Conversions API** → **Generate access token** → copy it (secret).
- ☐ Test Events tab → copy the **test event code** (e.g. `TEST12345`) for setup.

2 Add the client to Supabase

SQL editor → run once, replacing the 3 values

► Run SQL here: <https://supabase.com/dashboard/project/vkqmpmimpguymypxak/sql/new>

```
-- returns the client's id → use it in the webhook URL
insert into public.capi_clients (name, pixel_id, access_token, test_event_code)
values (
  'CLIENT NAME',
  'THEIR_PIXEL_ID',
  'THEIR_ACCESS_TOKEN',
  'THEIR_TEST_CODE'
)
returning id;
```

- ☐ Copy the returned **id** — this is the client's `{CLIENT_ID}`.

3 Landing page — head tracking code

Funnel → Settings → Tracking Code → Head

- ☐ Paste the **Meta Pixel base code** with the client's pixel id:

```
<script>
!function(f,b,e,v,n,t,s){if(f.fbq)return;n=f.fbq=function(){n.callMethod?
n.callMethod.apply(n,arguments):n.queue.push(arguments)};if(!f._fbq)f._fbq=n;
n.push=n;n.loaded=!0;n.version='2.0';n.queue=[];t=b.createElement(e);t.async=!0;
t.src=v;s=b.getElementsByTagName(e)[0];s.parentNode.insertBefore(t,s)}(window,
document,'script','https://connect.facebook.net/en_US/fbevents.js');
fbq('init', 'THEIR_PIXEL_ID');
fbq('track', 'PageView');
</script>
```

☐ Paste the capture script below it — universal, never changes between clients:

```
<script>
(function () {
  function getCookie(n) {
    var m = document.cookie.match('(^\s*)\\s*' + n + '\\s*=\\s*([^;]+)');
    return m ? m.pop() : '';
  }
  var params = new URLSearchParams(location.search);
  var fbclid = params.get('fbclid');
  var fbc = getCookie('_fbclid');
  if (!fbc && fbclid) fbc = 'fb.1.' + Date.now() + '.' + fbclid;

  var values = {
    fbc: fbc,
    fbp: getCookie('_fbp'),
    event_id: (window.crypto && crypto.randomUUID() ? crypto.randomUUID()
      : String(Date.now()) + '.' + Math.random()),
    user_agent: navigator.userAgent,
    ip: ''
  };

  function setValue(el, val) {
    var win = el.ownerDocument.defaultView;
    var setter = Object.getOwnPropertyDescriptor(
      win.HTMLInputElement.prototype, 'value'
    ).set;
    setter.call(el, val || '');
    el.dispatchEvent(new Event('input', { bubbles: true }));
    el.dispatchEvent(new Event('change', { bubbles: true }));
  }

  function allDocs(root){
    var list=[root];
    root.querySelectorAll('iframe').forEach(function(f){
      try{ if(f.contentDocument) list=list.concat(allDocs(f.contentDocument)); }catch(e){}
    });
    return list;
  }

  function apply() {
    allDocs(document).forEach(function (doc) {
      Object.keys(values).forEach(function (key) {
        if (!values[key]) return;
        doc.querySelectorAll('input[data-q="' + key + '"]').forEach(function (el) {
          if (el.value !== values[key]) setValue(el, values[key]);
        });
      });
    });
  }

  setInterval(apply, 600);

  fetch('https://api.ipify.org?format=json')
    .then(function (r) { return r.json(); })
    .then(function (d) { values.ip = d.ip; })
    .catch(function () {});
})();
</script>
```

The capture script matches fields by their `data-q="<name>"` attribute — that's why the exact field names in Step 4 matter, and what lets one script work for every client.

4

Form / survey — hidden fields

Add the browser-data fields

- ☐ Create 5 **custom fields** (Text), named **exactly**: `fbc` `fbp` `event_id` `user_agent` `ip`
- ☐ Add all 5 to the form as **hidden fields**.
- ☐ **Surveys**: place them on the **first step** so they render on load.
- ☐ Leave **Hidden value blank** and **not required** (fbc is empty for non-ad traffic).
- ☐ Map the visible **zip** field to the standard **Postal Code** contact field.

5

Workflow — fire the Lead

Automation → Workflows → Create Workflow

- ☐ Trigger: **Form Submitted** → filter to the specific form.
- ☐ Action: **Webhook** → Method `POST` → URL `.../api/capi/{CLIENT_ID}`
- ☐ Add the **Custom Data** rows below.
- ☐ Workflow **Settings** → enable **"Allow Re-Entry"**, then **Publish**.

KEY	VALUE	KEY	VALUE
event_name	Lead	fbc	{{contact.fbc}}
first_name	{{contact.first_name}}	fbp	{{contact.fbp}}
last_name	{{contact.last_name}}	event_id	{{contact.event_id}}
phone	{{contact.phone}}	user_agent	{{contact.user_agent}}
zip	{{contact.postal_code}}	ip	{{contact.ip}}

6

Test (while the test code is set)

Confirm capture, then a real submission

- ☐ On the live page, hard-refresh and run in the console to confirm capture:

```
[ 'fbp', 'ip', 'user_agent', 'event_id', 'fbc' ].map(function(k){
  var el=document.querySelector('input[data-q="'+k+'"]');
  return k+' = '+ (el?el.value:'?')+'';
});
```

- ☐ Submit with a **fresh name + phone**; keep **Meta Test Events** open (it only shows events arriving while watching).
- ☐ Confirm the Lead shows: First name, Last name, Phone, ZIP, Browser id (fbp), IP, User agent.

7

Go live

Make real leads count

► Run SQL here: <https://supabase.com/dashboard/project/vkqmpmimpguymypxak/sql/new>

```
update public.capi_clients set test_event_code = null
where id = 'THE_CLIENT_ID';
```

☐ Events now count as real conversions and appear in **Events / Overview** (delayed), not Test Events.

Only run this once testing is done. After it, every submission is a real conversion.

Troubleshooting

SYMPTOM	CAUSE	FIX
Event not in Test Events	Only shows events arriving while watching ; or the tab's code $\neq$ the stored one	Keep tab open; confirm the code matches the DB row
Hidden fields empty on contact	Fields not on step 1, or page not hard-refreshed	Put fields on first step; hard-refresh; check by <code>data-q</code>
Form submit doesn't fire workflow	GHL blocks workflow re-entry for an existing contact	Enable "Allow Re-Entry", or test with a new phone/name
Values won't "stick"	GHL forms are Vue-based; plain value set is ignored	Script fires input/change events (already handled)
permission denied for capi_clients	service_role lacks grant	<code>grant select on public.capi_clients to service_role;</code>
"Deduplicated" in Test Events	Same event_id received twice (repeat testing) — Meta protecting you	Fine — don't add a browser Lead event
Double-counted leads	A browser Lead + the server Lead	Remove all browser Lead snippets

Reference

MATCH PARAMETERS SENT	<code>ph</code> phone · <code>fn/ln</code> name · <code>zp</code> zip · <code>fbp</code> · <code>fbc</code> (ad traffic only) · <code>client_ip_address</code> · <code>client_user_agent</code> . Add <code>email</code> to the webhook if a client's form collects it → sends hashed <code>em</code> .
WEBHOOK FIELD ALIASES	Name: <code>first_name+last_name</code> or a single <code>full_name</code> (auto-split). Zip: <code>zip</code> , <code>zip_code</code> , or <code>postal_code</code> . Value (optional): <code>value</code> + <code>currency</code> — omit for value-free Leads.